Inheritance important Points :
----

* **Deriving** a new class (Child class) from existing class (Parent class) in such a way that the new class will **acquire all the features and properties of existing class is called Inheritance.**
[private members are not inherited because private members are available within the same class only]

* Inheritance is one of the most important feature of OOP, which provides **"Code Reusability"**.

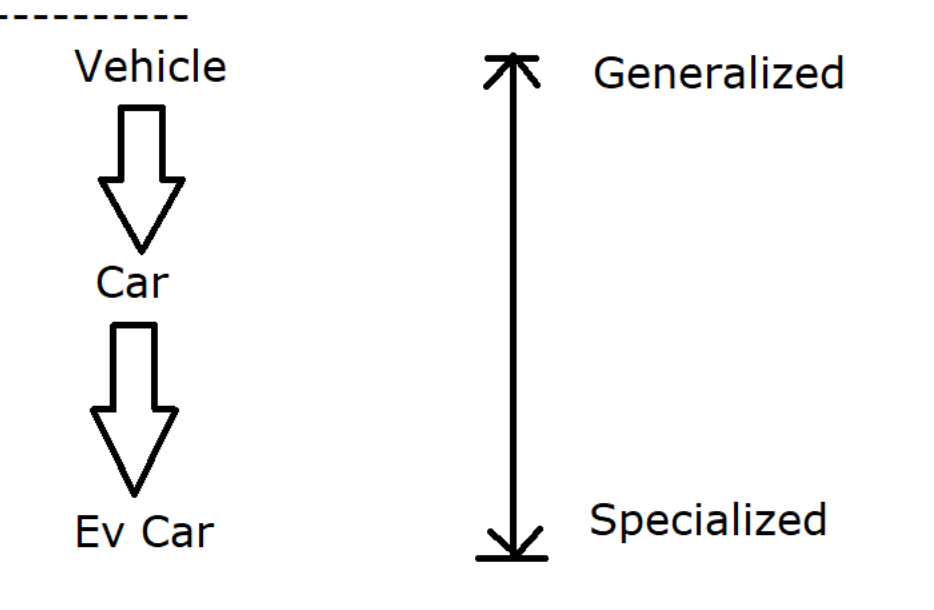
* We can achieve inheritance by using "extends" keyword.

* Using Inheritance mechanism, the relationship between the classes is, Parent & Child, According to Java, Parent class is called super class and child class is called sub class.

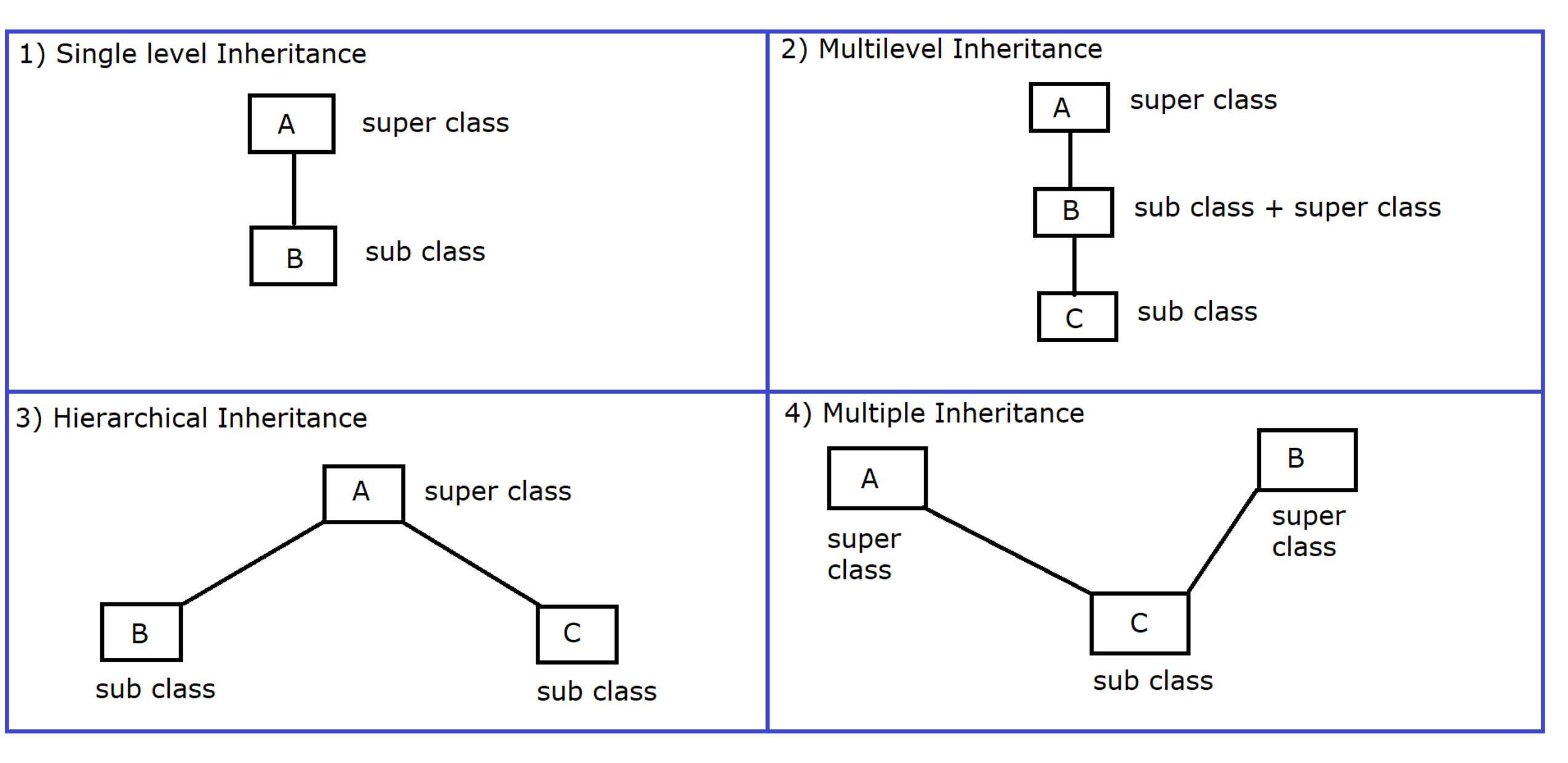
* Using Inheritance, all the properties (fields) & behaviour (methods) of super class are by default available to sub class so, sub class **need not to start the process** from begning onwards.

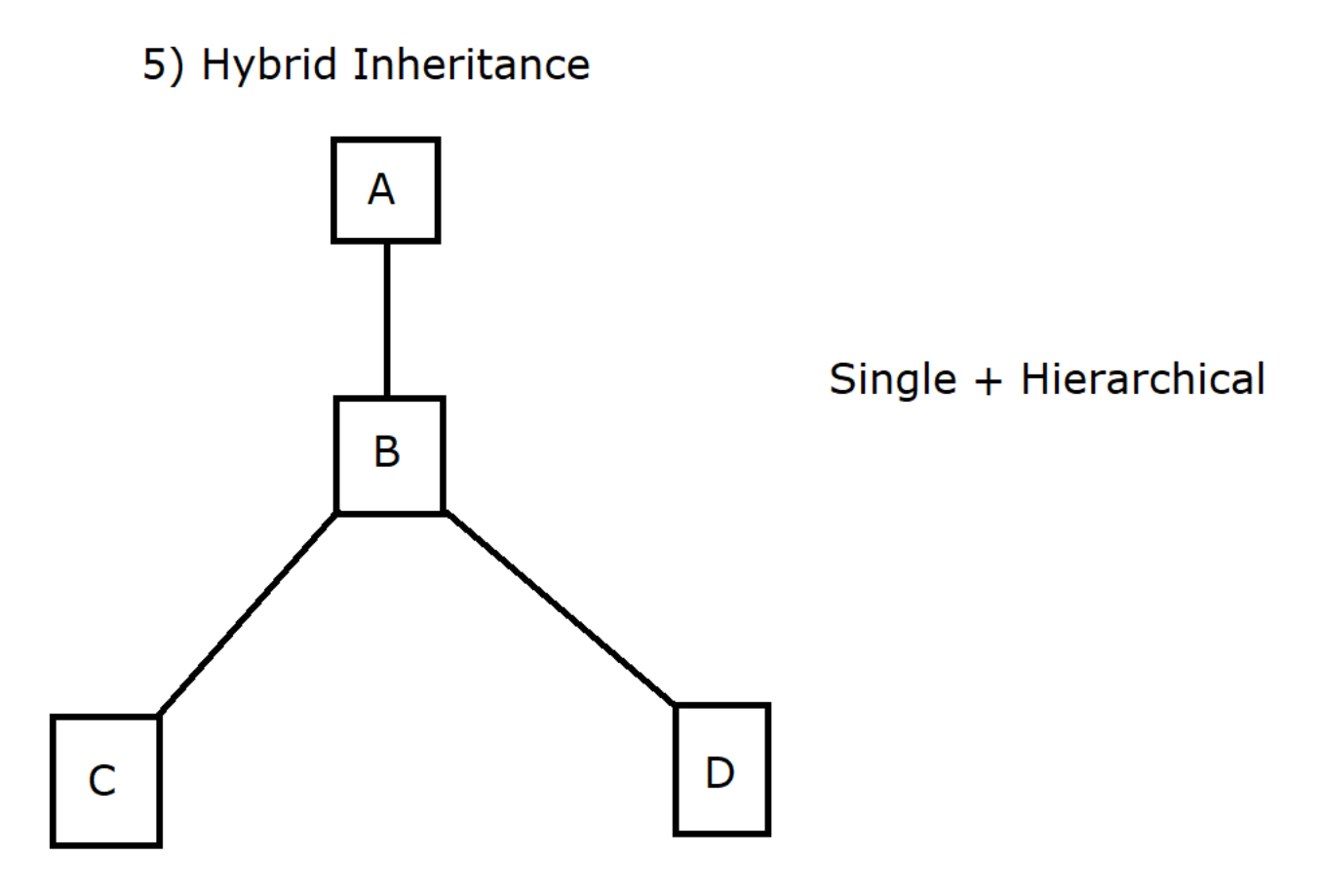
* Inheritance provides hierarchical classification of the classes, In this hierarchy, If we move towards upward direction then more generalized properties will occur but if we move towards downward direction then more specialized properties will occur.

Example :



Types of Inheritance :
----
5 types of Inheritance :
1) Single level Inheritance
2) Multilevel Inheritance
3) Hierarchical Inheritance
4) Multiple Inheritance [Not supported by java using classes]
5) Hybrid Inheritance [Combination of two]





Note : In Java, java.lang.Object class is by default, super class of all the classes in java

Test.java

```
public class Test
{

}
```
javac

Test.class

```
public class Test extends Object
{
  public Test()
  {
    super();
  }

}
```

Basic Program on Inheritance :

```
class Animal
{
     public void eat()
     {
          IO.println("Animal is eating");
     }
}
class Dog extends Animal
{

     public void bark()
     {
          IO.println("Dog is barking");
     }
}

public class InheritanceDemo
{
     public static void main(String[] args)
     {
          Dog dog = new Dog();
          dog.eat();
          dog.bark();
     }
}
```

In Inheritance, we should always create the object for specialized class

Rules
1)
javac
=====
Java compiler will always search the fields and methods based on the current reference (reference variable type)
Example :
Animal a = new Animal();
a.eat(); //a is of type Animal so, java compiler will serach the eat method in the Animal class

jvm
===
JVM will always execute the methods based on the current object.
Example:
Animal a = new Animal();
a.eat(); //a is holding the Object of Animal so, JVM will start executing from Animal class object

So, conclusion is : javac works on reference variable
JVM works on object.

2) javac (reference) & JVM (Object) both will search the members from bottom to top

---

Can we inherit constructor ?
----
* Constructors **are not inherited** in java that means super class constructor will not be available to sub class.

Example :

```
class Alpha
{
   public Alpha()
   {
   }
}
class Beta  extends Alpha
{
   public Beta()
   {
     super();

   }
}


      Beta beta = new Beta();
```

[If constructor will be available in the sub class then it will not match with sub class name]

```
package com.ravi.inheritance_with_encapsulation;

class Alpha
{
    private int x;
    private int y;

    public void setX(int x)
    {
        this.x = x;
    }
    public void setY(int y)
    {
        this.y = y;
    }

    public int getY()
    {
        return this.y;
    }

    public int getX() {
        return x;
    }
}
class Beta extends Alpha
{
    public void showData()
    {
        IO.println("x value is :"+getX());
        IO.println("y value is :"+getY());
    }
}

public class InheritanceWithEncapsulation
{
    public static void main(String[] args)
    {
        Beta beta = new Beta();
        beta.setX(100);
        beta.setY(200);
        beta.showData();

    }

}
```